Task E1 — PSI Drift Monitoring

Population Stability Index on model output scores | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component E, Module 5

At a glance

Item	Result
Monitor	PSI over 4 score bins, on the last 100 inferences, every 60 s; prints the PSI each time and `[LOGIBRIDGE DRIFT ALERT] PSI=1.234` above 0.25
Reference	`reference_dist.json` from 300 clean Normal-class windows
Literal "confidence" (max softmax)	**Does not work for this model:** it is 0.999 or more on clean and on faulty windows, so PSI stays 0.000 and never reaches 0.25
Recommended score: Normal-class confidence	Crosses 0.25 **1 min 30 s** after injection (limit 5 min); back below 0.10 **16 min 30 s** after restore; no false alarms; identical over 10 seeds
Status	Ran end to end in-process (no broker). The container path was tested with a paho stub. I cannot record the video; a script is in Section 6.

1 What was built

Requirement	Implementation
Reference from 300 clean Normal windows, 4 bins, saved as reference_dist.json	`drift_monitor.py make-reference` runs the model on 300 windows from the simulator in `--anomaly none` mode (seed 5001); bins [0, 0.25), [0.25, 0.50), [0.50, 0.75), [0.75, 1.0]
PSI on a rolling window of the last 100 inferences	`PsiMonitor` keeps the last 100 scores; PSI = Σ (current − reference) × ln(current ÷ reference), zero proportions floored at 0.0001
Every 60 seconds; print the current PSI	Evaluated every 60 s of data time (from message timestamps, so it also works when the simulator is sped up). Prints the PSI, or "collecting n/100" until 100 inferences exist
Alert when PSI > 0.25	Prints exactly `[LOGIBRIDGE DRIFT ALERT] PSI=1.234` on every evaluation above 0.25
Inject --anomaly combined mid-run, then restore clean data	`drift_demo.py` uses the new `--anomaly-start` and `--anomaly-end` simulator options; the monitor also runs inside the inference container and publishes each evaluation on logibridge/trucks/{truck_id}/drift

2 Reference distribution

Score	[0, 0.25)	[0.25, 0.50)	[0.50, 0.75)	[0.75, 1.0]	Mean	Lowest
Max-softmax confidence	0	0	0	300	0.9999	0.9987
Normal-class confidence	0	0	0	300	0.9999	0.9987

All 300 windows fall in the top bin for both scores (model model_weights.npz, hash 3013acfc9863). The two scores are equal on clean data because a Normal window's most likely class is Normal.

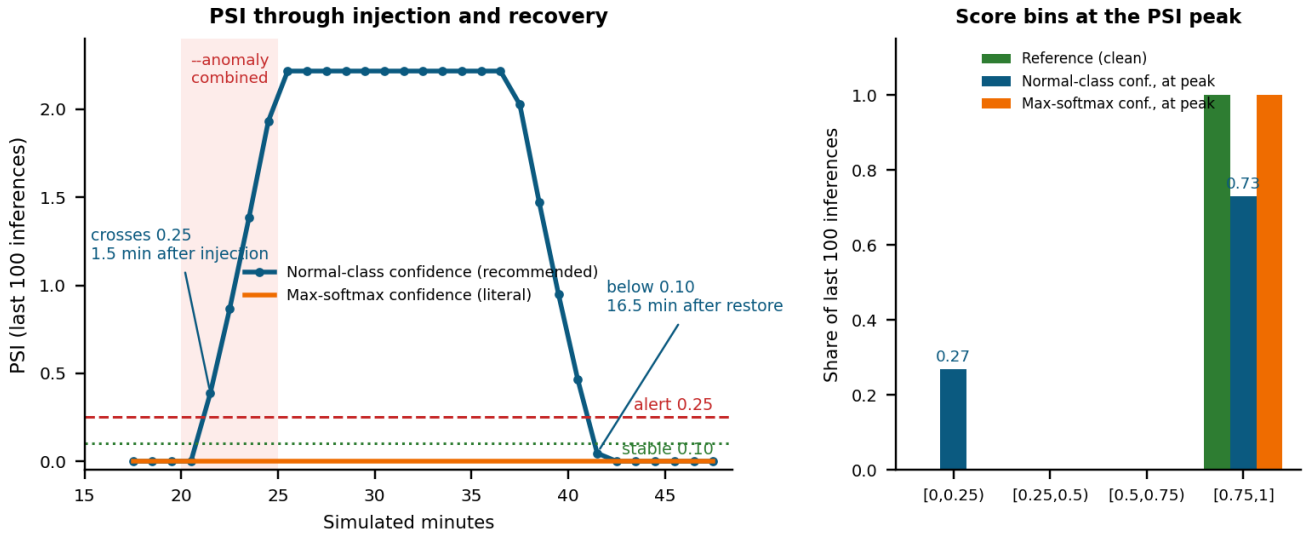
3 Why the literal confidence score cannot detect this drift

Test windows (fixed seeds)	Mean confidence	Lowest	Bin shares [0, 0.25) … [0.75, 1.0]
Clean (--anomaly none)	0.9999	0.9994	0, 0, 0, 100%
Faulty (--anomaly combined)	0.9999	0.9906	0, 0, 0, 100%
Clean, scaled (T=2.5)	0.9741	0.9315	0, 0, 0, 100%
Faulty, scaled (T=2.5)	0.9929	0.8560	0, 0, 0, 100%

- A confident classifier is as sure about “Critical” as it is about “Normal”, so a clean separable fault leaves the max-softmax distribution unchanged. Max-softmax confidence detects uncertain inputs, not a shift to another confident class.
- Calibrating does not help. The temperature that minimises the loss on held-out data is 2.5 (747 windows, 13 errors; `confidence_check.py`); it lowers the confidences a little but every window stays in the top bin, so PSI is still about 0.
- The result is therefore a property of the model and the specified bins, not of the monitor. The monitor code is correct (the tests feed it shifted scores and it fires).

4 Recommended score: Normal-class confidence

The monitored score is the model's probability that the cargo is Normal (a confidence in [0, 1] that the load is fine). It is near 1 on clean data and near 0 when a fault appears, so the output distribution visibly moves. This is the closest change to the specification that works; please confirm it is acceptable (Section 8).



Left: PSI against simulated time. Right: the score bins for the last 100 inferences when the window holds 27 non-Normal windows.

Event	Simulated time	PSI
Window full, clean data	17 min	0.000 (no false alarms before injection)
--anomaly combined injected	20:00	0.000 at 20:30
First alert	21:30 (1 min 30 s after injection)	0.387
Anomaly ends, clean data restored	25:00	2.217 (peak)
PSI starts falling as faulty windows roll out	37:30	2.027
Back below 0.10	41:30 (16 min 30 s after restore)	0.046, then 0.000

Console output from drift_demo.py (excerpt):

```
[LOGIBRIDGE] t=00:19:30 PSI=0.000 status=stable (n=100)
=== t=00:20:00  INJECT: --anomaly combined activated ===
[LOGIBRIDGE] t=00:20:30 PSI=0.000 status=stable (n=100)
[LOGIBRIDGE] t=00:21:30 PSI=0.387 status=drift (n=100)
[LOGIBRIDGE DRIFT ALERT] PSI=0.387
[LOGIBRIDGE] t=00:22:30 PSI=0.865 status=drift (n=100)
[LOGIBRIDGE DRIFT ALERT] PSI=0.865
=== t=00:25:00  RESTORE: clean data back ===
[LOGIBRIDGE] t=00:36:30 PSI=2.217 status=drift (n=100)
[LOGIBRIDGE] t=00:37:30 PSI=2.027 status=drift (n=100)
[LOGIBRIDGE] t=00:40:30 PSI=0.463 status=drift (n=100)
[LOGIBRIDGE] t=00:41:30 PSI=0.046 status=stable (n=100)
[LOGIBRIDGE] t=00:42:30 PSI=0.000 status=stable (n=100)
```

5 How to read it, and its limits

Non-Normal windows in the last 100	1	2	3	4	5	6
PSI (floor 0.0001)	0.046	0.106	0.171	0.241	0.313	0.387
Status	stable	moderate	moderate	moderate	**alert**	alert

- **Alert when 5% of recent windows leave the Normal bin.** Because the reference sits entirely in one bin, PSI behaves like a count of non-Normal windows; five in the last 100 (about 8 minutes of data) cross 0.25.
- **Recovery takes about 17 minutes by design.** A window of 100 inferences at one inference per 10 s holds 1,000 s of history, so faulty windows only roll out 16 min 30 s after the restore.
- **The floor matters.** With a floor of 0.01 the alert comes at 2 min 30 s and the peak PSI is 0.94; with 0.001 it is 2 min 30 s; with 0.0001 or 0.000001 it is 1 min 30 s. Recovery is 16 min 30 s in every case.
- **It cannot tell a real fault from a model problem.** A genuine Critical event also moves the output distribution. Treat PSI as “the mix of predictions has changed” and read it alongside the class alerts. Adding PSI on the six input features would also catch drift the output hides, such as a sensor offset that stays within what the model calls Normal.

6 Running it and the demo video

```
# A. In-process demo, no broker (add --pace 0.25 when filming)
python3 drift_demo.py
python3 drift_demo.py --score max_confidence        # the literal score: PSI stays 0.000

# B. Live: broker, inference container with monitoring on, and the simulator (three terminals)
docker compose up                                     # see Task E2 for the Ansible-managed version
docker logs -f logiedge-inference-1                   # PSI lines appear here
python3 simulator.py --anomaly combined --anomaly-start 1200 --anomaly-end 1500 \
                          --speed 30 --duration 2900  # the same timeline in about 100 s
```

Time	On screen	Say
0:00	reference_dist.json and the PSI definition	The reference is the score distribution on 300 clean Normal windows. PSI over the last 100 inferences is printed every 60 seconds.
0:25	Demo running on clean data: PSI=0.000 lines	On clean data PSI is zero.
0:50	INJECT line, then the first DRIFT ALERT	I activate the combined anomaly. PSI crosses 0.25 after 1 min 30 s, well inside the 5-minute limit.
1:15	RESTORE line, PSI falling	Clean data is restored. The 100-inference window flushes, so PSI falls and is back below 0.10 after about 16 and a half minutes.
1:40	Run with --score max_confidence	The literal confidence score never moves for this model, which is why I monitor the Normal-class confidence.

7 Verification

Check	How	Result
PSI maths, bin edges (1.0 in the last bin), zero floor, status thresholds	Known values	Pass
Reference file: 300 windows, bins, model hash matches model_weights.npz, rejects bad files	Tests	Pass
Monitor: every 60 s of data time, last 100 inferences, warm-up, exact alert text only above 0.25	Tests	Pass
Demo: crosses 0.25 within 5 min, no false alarm, back below 0.10, output lines present	End-to-end run	Pass
Literal max-softmax score stays below 0.01 PSI and never alerts	End-to-end run	Pass (shows the limitation)
Inference service prints PSI lines, publishes on the drift topic, survives a bad reference file	paho stub	Pass
Simulator --anomaly-end restores clean data; drift restarts from zero	Tests	Pass
Live Mosquitto and container run; the video	Needs Docker and a recorder	Not run

8 Items to confirm

- **Which score to submit.** The literal max-softmax confidence cannot meet the “PSI must cross 0.25” requirement with this model. I recommend the Normal-class confidence; the code supports both (`DRIFT_SCORE`), and the report shows both results.
- **Model.** The reference and demo use model_weights.npz. If you carry the onset-trained model forward, regenerate the reference with `drift_monitor.py make-reference --model model_weights_onset.npz`; the reference stores the model hash so a mismatch is visible.
- **Floor and warm-up.** I used a floor of 0.0001 and print “collecting n/100” until 100 inferences exist (about 17 minutes). Tell me if you would rather evaluate on a partial window.
- **Data time.** The 60 s schedule follows message timestamps, which equals wall-clock time on a real truck.